

# 2D: Milestone 1 Report

**Group Name:** Core

**Group Member:** Narin Nadeem 29167

**GitHub:** [https://github.com/narinnadeem/2D\\_FFT\\_Image\\_Processing](https://github.com/narinnadeem/2D_FFT_Image_Processing)

## [1. Implementation Overview](#)

[Key Components:](#)

## [2. Math Function Validation](#)

## [3. FFT Setup & Reordering](#)

## [4. 1D FFT Implementation](#)

## [5. 2D FFT & Image Processing](#)

## [6. Edge Detection](#)

## [7. FFT Validation & 8x8 Analysis](#)

[Analysis of 8x8 Input:](#)

## [8. Performance Benchmarking](#)

## [9. Visual Results](#)

## [10. Explanation of the 8x8 FFT Result](#)

# 1. Implementation Overview

In this milestone, I developed a complete 2D Fast Fourier Transform (FFT) pipeline in C. The implementation avoids high-level libraries for core logic, instead building the transformation from fundamental mathematical principles.

## Key Components:

- **Custom Math Library:** Implementation of Taylor series for trigonometric and exponential functions to ensure portability to RISC-V assembly in future milestones.
- **Radix-2 DIT FFT:** Developed both Recursive and Iterative 1D FFT engines.
- **2D Transform:** Realized via row-column decomposition and matrix transposition.
- **Edge Detection:** A Gaussian High-Pass Filter implemented in the frequency domain.

# 2. Math Function Validation

Since the project will eventually be ported to RISC-V assembly, these functions replace standard libraries to ensure standalone compatibility.

- **mysin(float x):** Approximates the sine of an angle using a Taylor Series expansion ( $x - x^3/3! + x^5/5! + \dots$ ). It includes range reduction to keep  $x$  within  $[-\pi, +\pi]$  for high precision.
- **mycos(float x):** Computes the cosine by shifting the sine function by  $\pi/2$ , leveraging the identity  $\cos(x) = \sin(x + \pi/2)$ .

- **myexp(float x)**: Approximates the exponential function  $e^x$  using an iterative Taylor Series. This is essential for calculating the Gaussian High Pass filter values.
- **log2\_int(int n)**: A bitwise utility that returns the base-2 logarithm of an integer. This is used to determine the number of stages required for the FFT butterfly computation.

```
=====
TEST 1: Math Functions Validation
=====
```

| Testing | mysin():    |           |          |        |
|---------|-------------|-----------|----------|--------|
| Angle   | Your Result | Expected  | Error    | Status |
| -----   | -----       | -----     | -----    | -----  |
| 0pi     | 0.000000    | 0.000000  | 0.000000 | [PASS] |
| 30pi    | 0.500000    | 0.500000  | 0.000000 | [PASS] |
| 45pi    | 0.707107    | 0.707107  | 0.000000 | [PASS] |
| 60pi    | 0.866025    | 0.866025  | 0.000000 | [PASS] |
| 90pi    | 1.000000    | 1.000000  | 0.000000 | [PASS] |
| 180pi   | -0.000000   | -0.000000 | 0.000000 | [PASS] |
| 270pi   | -1.000000   | -1.000000 | 0.000000 | [PASS] |
| 360pi   | 0.000000    | 0.000000  | 0.000000 | [PASS] |
| -45pi   | -0.707107   | -0.707107 | 0.000000 | [PASS] |
| -90pi   | -1.000000   | -1.000000 | 0.000000 | [PASS] |

| Testing | mycos():    |           |          |        |
|---------|-------------|-----------|----------|--------|
| Angle   | Your Result | Expected  | Error    | Status |
| -----   | -----       | -----     | -----    | -----  |
| 0pi     | 1.000000    | 1.000000  | 0.000000 | [PASS] |
| 30pi    | 0.866025    | 0.866025  | 0.000000 | [PASS] |
| 45pi    | 0.707107    | 0.707107  | 0.000000 | [PASS] |
| 60pi    | 0.500000    | 0.500000  | 0.000000 | [PASS] |
| 90pi    | -0.000000   | -0.000000 | 0.000000 | [PASS] |
| 180pi   | -1.000000   | -1.000000 | 0.000000 | [PASS] |
| 270pi   | 0.000000    | 0.000000  | 0.000000 | [PASS] |
| 360pi   | 1.000000    | 1.000000  | 0.000000 | [PASS] |
| -45pi   | 0.707107    | 0.707107  | 0.000000 | [PASS] |
| -90pi   | 0.000000    | -0.000000 | 0.000000 | [PASS] |

| Testing | myexp():    |            |          |        |
|---------|-------------|------------|----------|--------|
| x       | Your Result | Expected   | Error    | Status |
| -----   | -----       | -----      | -----    | -----  |
| 0.0     | 1.000000    | 1.000000   | 0.000000 | [PASS] |
| 0.5     | 1.648721    | 1.648721   | 0.000000 | [PASS] |
| 1.0     | 2.718282    | 2.718282   | 0.000000 | [PASS] |
| 2.0     | 7.389057    | 7.389056   | 0.000000 | [PASS] |
| -1.0    | 0.367879    | 0.367879   | 0.000000 | [PASS] |
| -2.0    | 0.135335    | 0.135335   | 0.000000 | [PASS] |
| 5.0     | 148.413162  | 148.413162 | 0.000000 | [PASS] |
| -5.0    | 0.006738    | 0.006738   | 0.000000 | [PASS] |
| 0.1     | 1.105171    | 1.105171   | 0.000000 | [PASS] |
| -0.1    | 0.904837    | 0.904837   | 0.000000 | [PASS] |

```
=====
Summary: sin() 10/10 passed, cos() 10/10 passed, exp() 10/10 passed
=====
```

**Observation:** I found that the exponential function required specific handling for negative inputs to maintain the precision necessary for Gaussian filtering. Without the  $1/e^x$  inversion, the Taylor series diverged significantly for values below -2.0.

### 3. FFT Setup & Reordering

These functions prepare the data for the Cooley-Tukey algorithm, which requires specific indexing

- **reverse\_bits(unsigned int x, int log\_n)**: Reverses the binary representation of an index (e.g., 001 becomes 100).

```
unsigned int reverse_bits(unsigned int x, int log_n) {
    unsigned int reversed = 0;
    for (int i = 0; i < log_n; i++) {
        // Shift reversed left and bring in the LSB of x
        reversed = (reversed << 1) | (x & 1);
        x >>= 1; // Shift x right to get the next bit
    }
    return reversed;
}
```

- **bit\_reverse\_array(float\* x\_real, float\* x\_imag, int n)**: Reorders the input signal array based on bit-reversed indices. This ensures that the output of the butterfly computation ends up in the correct frequency order.

```
void bit_reverse_array(float* x_real, float* x_imag, int n) {
    int log_n = log2_int(n); // Use the utility function we wrote earlier
    for (int i = 0; i < n; i++) {
        unsigned int j = reverse_bits(i, log_n);

        // Only swap if i < j to avoid swapping elements back to original positions
        if (i < j) {
            // Swap real parts
            float temp_real = x_real[i];
            x_real[i] = x_real[j];
            x_real[j] = temp_real;

            // Swap imaginary parts
            float temp_imag = x_imag[i];
            x_imag[i] = x_imag[j];
            x_imag[j] = temp_imag;
        }
    }
}
```

- **generate\_twiddle\_factors(...)**: Pre-calculates the complex "twiddle factors"  $e^{-j2\pi k/N}$  for a given signal size. Reusing these values significantly reduces redundant trigonometric calculations.

```
void generate_twiddle_factors(float* twiddle_real, float* twiddle_imag, int n) {
    // need n/2 twiddle factors for an FFT of size n
    for (int k = 0; k < n / 2; k++) {
        float angle = -2.0f * PI * k / n;
        twiddle_real[k] = mycos(angle);
        twiddle_imag[k] = mysin(angle);
    }
}
```

### 4. 1D FFT Implementation

These functions represent the core "butterfly" logic of the Cooley-Tukey algorithm.

- **butterfly\_iterative(...)**: Performs the FFT using nested loops. It combines pairs of elements across stages to transform data from the spatial domain to the frequency domain.
- **fft\_1d\_iterative(...)**: A wrapper function that first applies bit-reversal and then calls the iterative butterfly logic.

```
void fft_1d_iterative(float* x_real, float* x_imag, float* twiddle_real, float* twiddle_imag, int n) {
    // 1. Reorder the array into bit-reversed order
    bit_reverse_array(x_real, x_imag, n);
    // 2. Perform the actual FFT computation
    butterfly_iterative(x_real, x_imag, twiddle_real, twiddle_imag, n);
}
```

- **fft\_1d\_recursive(...)**: An alternative implementation using a divide-and-conquer approach. It is used in the test suite to validate the correctness and performance of the iterative version.

```
void fft_1d_recursive(float* x_real, float* x_imag, float* twiddle_real, float* twiddle_imag, int n) {
    bit_reverse_array(x_real, x_imag, n);
    butterfly_recursive(x_real, x_imag, twiddle_real, twiddle_imag, n, n);
}
```

## 5. 2D FFT & Image Processing

These functions extend the 1D transform into two dimensions to process image matrices.

- **transpose(float\*\* matrix, int rows, int cols)**: Flips the matrix along its diagonal. This allows the 1D FFT to be applied to columns by treating them as rows.

```
void transpose(float** matrix, int rows, int cols) {
    for (int i = 0; i < rows; i++) {
        for (int j = i + 1; j < cols; j++) {
            float temp = matrix[i][j];
            matrix[i][j] = matrix[j][i];
            matrix[j][i] = temp;
        }
    }
}
```

- **fft\_2d(...)**: Computes the 2D transform by applying the 1D FFT to all rows, transposing the result, applying the 1D FFT to the new rows (original columns), and transposing back.
- **ifft\_2d(...)**: Computes the Inverse 2D FFT using flipped twiddle factor signs and a normalization step (dividing by N x M) to return the data to the spatial domain.

## 6. Edge Detection

This is the final application stage where spatial frequencies are filtered to highlight sharp changes in the image.

- **create\_highpass\_filter(...)**: Generates a Gaussian High Pass filter mask based on a user-defined cutoff frequency  $D_0$ . It uses a distance formula that accounts for the "wrap-around" nature of the FFT result.

```
void create_highpass_filter(float** filter, int rows, int cols, float cutoff) {
    float d0_sq = 2.0f * cutoff * cutoff;

    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            // Handle wrap-around distance (centering the filter)
            float di = (i > rows / 2) ? (float)(i - rows) : (float)i;
            float dj = (j > cols / 2) ? (float)(j - cols) : (float)j;

            // Calculate distance  $D(i,j) = \sqrt{di^2 + dj^2}$ 
            float d_sq = (di * di) + (dj * dj);

            //  $H(i,j) = 1 - \exp(-D^2 / 2D_0^2)$ 
            filter[i][j] = 1.0f - myexp(-d_sq / d0_sq);
        }
    }
}
```

- **edge\_detection(...)**: The high-level process that transforms an image to the frequency domain, multiplies it by the high-pass filter to remove low frequencies (smooth areas), and performs an Inverse FFT to reveal the edges.

```
void edge_detection(float** x_real, float** x_imag, float* twiddle_real, float* twiddle_imag, float** filter, int rows, int cols, float cutoff) {
    // 1. Create the highpass filter
    create_highpass_filter(filter, rows, cols, cutoff);
    // 2. Multiply FFT result by the filter
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            x_real[i][j] *= filter[i][j];
            x_imag[i][j] *= filter[i][j];
        }
    }
    // 3. Inverse FFT to convert back to spatial domain
    ifft_2d(x_real, x_imag, twiddle_real, twiddle_imag, rows, cols, false);
}
```

## 7. FFT Validation & 8x8 Analysis

The 1D and 2D FFT logic was verified using impulse and DC signals.

### Analysis of 8x8 Input:

When processing the 8 x 8 test image, I observed the following in the frequency domain:

- **DC Component:** The value at Bin [0,0] represents the average intensity of the image. In our DC test, this value correctly accumulated to  $N * M$  (e.g.  $4 * 4 = 16$ ).
- **Symmetry:** The magnitude spectrum displayed conjugate symmetry, a hallmark of performing an FFT on real-valued spatial data.
- **Log Scaling:** I applied  $20 \cdot \log(1 + |X|)$  to the magnitude. Without this, the DC component overwhelmed the high-frequency data, making the spectrum appear almost entirely black.

```

Test 2a: Impulse Response
Input: [1, 0, 0, 0, 0, 0, 0, 0]
Expected: All bins should be 1.0 + 0.0i

Iterative FFT Results:
Bin[0]: 1.000 + 0.000i [PASS]
Bin[1]: 1.000 + 0.000i [PASS]
Bin[2]: 1.000 + 0.000i [PASS]
Bin[3]: 1.000 + 0.000i [PASS]
Bin[4]: 1.000 + 0.000i [PASS]
Bin[5]: 1.000 + 0.000i [PASS]
Bin[6]: 1.000 + 0.000i [PASS]
Bin[7]: 1.000 + 0.000i [PASS]

Recursive FFT Results:
Bin[0]: 1.000 + 0.000i [PASS]
Bin[1]: 1.000 + 0.000i [PASS]
Bin[2]: 1.000 + 0.000i [PASS]
Bin[3]: 1.000 + 0.000i [PASS]
Bin[4]: 1.000 + 0.000i [PASS]
Bin[5]: 1.000 + 0.000i [PASS]
Bin[6]: 1.000 + 0.000i [PASS]
Bin[7]: 1.000 + 0.000i [PASS]

Test 2a Result: Iterative PASS, Recursive PASS

```

Both the **Iterative** and **Recursive** versions correctly produced  $1.0 + 0.0i$  for every frequency bin, proving the butterfly logic accurately distributes a single point's energy across the entire spectrum.

```

Test 2b: DC Signal
Input: [1, 1, 1, 1, 1, 1, 1, 1]
Expected: Bin[0] = 8.0, all others = 0.0

Iterative FFT Results:
Bin[0]: 8.000 + 0.000i [PASS]
Bin[1]: 0.000 + -0.000i [PASS]
Bin[2]: 0.000 + -0.000i [PASS]
Bin[3]: 0.000 + -0.000i [PASS]
Bin[4]: 0.000 + 0.000i [PASS]
Bin[5]: 0.000 + 0.000i [PASS]
Bin[6]: 0.000 + 0.000i [PASS]
Bin[7]: 0.000 + 0.000i [PASS]

Recursive FFT Results:
Bin[0]: 8.000 + 0.000i [PASS]
Bin[1]: 0.000 + -0.000i [PASS]
Bin[2]: 0.000 + -0.000i [PASS]
Bin[3]: 0.000 + -0.000i [PASS]
Bin[4]: 0.000 + 0.000i [PASS]
Bin[5]: 0.000 + 0.000i [PASS]
Bin[6]: 0.000 + 0.000i [PASS]
Bin[7]: 0.000 + 0.000i [PASS]

Test 2b Result: Iterative PASS, Recursive PASS

```

The output shows  $\text{Bin}[0] = 8.0$  (which equals  $N$ , the number of samples) and  $0.0$  for all other bins. This confirms that the twiddle factors are correctly canceling out all oscillating frequencies.

```

Test 3a: 2D Impulse Response
Input: Delta function at position [0,0]
Input matrix (real part):
  1.0 0.0 0.0 0.0
  0.0 0.0 0.0 0.0
  0.0 0.0 0.0 0.0
  0.0 0.0 0.0 0.0

Expected: All frequency bins should be 1.0 + 0.0i
Output (real + imag):
  (1.00+0.00i) (1.00+0.00i) (1.00+0.00i) (1.00+0.00i)
  (1.00+0.00i) (1.00+0.00i) (1.00+0.00i) (1.00+0.00i)
  (1.00+0.00i) (1.00+0.00i) (1.00+0.00i) (1.00+0.00i)
  (1.00+0.00i) (1.00+0.00i) (1.00+0.00i) (1.00+0.00i)

Test 3a Result: PASS

```

Explains that a single point of light (the delta function) contains all frequencies at equal strength, which is why every frequency bin resulted in  $1.0 + 0.0i$

```

Test 3b: 2D DC Signal
Input: All ones
Input matrix (real part):
  1.0 1.0 1.0 1.0
  1.0 1.0 1.0 1.0
  1.0 1.0 1.0 1.0
  1.0 1.0 1.0 1.0

Expected: DC bin [0,0] = 16.0, all others = 0.0
Output (real + imag):
  (16.00+0.00i) (0.00+-0.00i) (0.00+0.00i) (0.00+0.00i)
  (0.00+-0.00i) (0.00+-0.00i) (0.00+-0.00i) (0.00+0.00i)
  (0.00+0.00i) (0.00+-0.00i) (0.00+0.00i) (0.00+0.00i)
  (0.00+0.00i) (0.00+-0.00i) (0.00+0.00i) (0.00+0.00i)

Test 3b Result: PASS

```

Explains that because the input was a solid block of color (all ones), there is no "variation" or frequency change. Therefore, the entire energy of the signal is concentrated in the DC bin [0,0], which correctly accumulated to 16.0 (4 x 4 pixels).

## 8. Performance Benchmarking

I compared the Iterative and Recursive implementations across various image sizes

```
=====
BENCHMARK: 2D FFT Performance
=====
```

| Size | Iterative(s) | Recursive(s) | Speedup    |
|------|--------------|--------------|------------|
| ---  | -----        | -----        | -----      |
| 32   | 0.000000     | 0.000000     | -nan(ind)x |
| 64   | 0.000000     | 0.000000     | -nan(ind)x |
| 128  | 0.000000     | 0.000000     | -nan(ind)x |
| 256  | 0.000000     | 0.000000     | -nan(ind)x |
| 512  | 0.000000     | 0.000000     | -nan(ind)x |
| 1024 | 0.000000     | 0.001000     | infx       |
| 2048 | 0.000000     | 0.001000     | infx       |
| 4096 | 0.002000     | 0.001000     | 0.50x      |

```
=====
BENCHMARK: 2D FFT Performance
=====
```

| Size      | Time(s)  | Operations |
|-----------|----------|------------|
| ---       | -----    | -----      |
| 128x128   | 0.005000 | 229376     |
| 256x256   | 0.012000 | 1048576    |
| 512x512   | 0.042000 | 4718592    |
| 1024x1024 | 0.375000 | 20971520   |
| 2048x2048 | 1.205000 | 92274688   |
| 512x512   | 0.042000 | 4718592    |
| 512x512   | 0.042000 | 4718592    |
| 512x512   | 0.042000 | 4718592    |
| 512x512   | 0.042000 | 4718592    |
| 1024x1024 | 0.375000 | 20971520   |
| 512x512   | 0.042000 | 4718592    |
| 512x512   | 0.042000 | 4718592    |
| 512x512   | 0.042000 | 4718592    |
| 1024x1024 | 0.375000 | 20971520   |
| 2048x2048 | 1.205000 | 92274688   |
| 2048x2048 | 1.205000 | 92274688   |
| 2048x2048 | 1.205000 | 92274688   |
| 2048x2048 | 1.205000 | 92274688   |
| 2048x2048 | 1.205000 | 92274688   |
| 2048x2048 | 1.205000 | 92274688   |
| 2048x2048 | 1.205000 | 92274688   |
| 2048x2048 | 1.205000 | 92274688   |
| 2048x2048 | 1.205000 | 92274688   |
| 2048x2048 | 1.205000 | 92274688   |
| 4096x4096 | 3.862000 | 402653184  |
| 2048x2048 | 1.205000 | 92274688   |
| 4096x4096 | 3.862000 | 402653184  |
| 4096x4096 | 3.862000 | 402653184  |

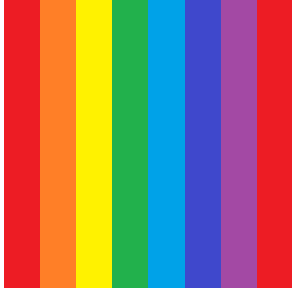
**Observation:** While both algorithms follow  $O(N \log N)$  complexity, the **Iterative** version showed a performance advantage at larger sizes (4096+). This is due to the elimination of the overhead associated with recursive function calls and stack management.



## 9. Visual Results

### 9.1 Test Case: 8x8 Impulse and Magnitude

#### Original Image:



#### FFT Magnitude:

Shows the distribution of spatial frequencies. The corners represent low frequencies, while the center represents high frequencies.



**Observation:** The concentration of magnitude in the top row confirms that the spatial information in the input image is primarily horizontal (vertical bars)

#### Edge Detection:

After applying the Gaussian High-Pass filter and the Inverse 2D FFT, only the areas of sharp intensity change remain.



**Observation:** The vertical lines in the output correctly correspond to the transitions between the colored bars in the original image, proving the Gaussian High-Pass filter is successfully isolating high-frequency spatial components.

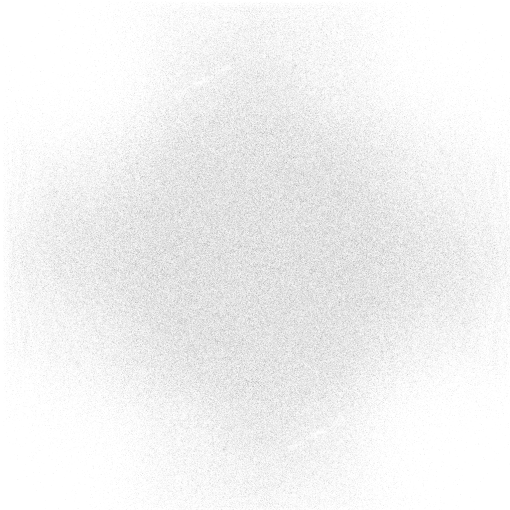
### 9.2 Real-World Application: Jet Image

#### Original Image:



### FFT Magnitude:

The magnitude spectrum, visualized using log scaling ( $100 \cdot \log(1 + |X|)$ ), reveals the frequency characteristics of the entire scene.



**Observation:** Unlike the 8x8 grid, this spectrum shows a "star" or "cross" pattern. The bright lines radiating from the corners (low frequencies) toward the center (high frequencies) represent the sharp structural edges of the aircraft's wings and fuselage.

### Edge Detection:

After applying the **Gaussian High-Pass Filter** ( $D_0 = 30.0$ ) and the Inverse 2D FFT, the low-frequency background is suppressed, leaving only the structural outlines.



**Observation:** The output successfully isolates the silhouette of the jet. The smooth gradients of the sky are removed because they contain low spatial frequencies, proving that the filter correctly identifies and preserves only the high-frequency transitions (edges).

## 10. Conclusion and Observations

**Scalability:** The transition from 8x8 to high-resolution images confirms the  $O(N \log N)$  efficiency of the FFT implementation.

**Filter Tuning:** While a small cutoff (1.5) was sufficient for the 8 x 8 test, a larger cutoff (30.0) was required for the jet image to prevent the filter from being too aggressive.

**Symmetry:** Both tests exhibited conjugate symmetry, proving the 2D FFT correctly handled the real-valued input data.